Chain of Responsibility

Chain · Route support tickets billing→tech→general · Build with Waqar

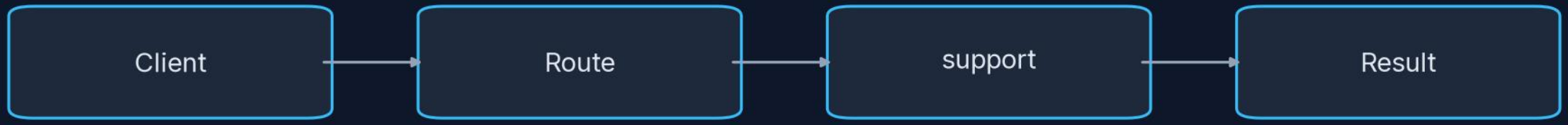
Today's Hook

- Problem: Route support tickets billing→tech→general
- Pattern: Chain — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Route support tickets billing→tech→general
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Chain: Route support tickets billing→tech→general

Real-World Scenario

- Running example: Route support tickets billing→tech→general
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.chain;

import com.buildwithwaqar.patterns.gof.behavioral.chain.*;
/** Problem: route support tickets without a giant if/else in the controller. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Chain problem: support ticket routing ===");
    var root = new BillingHandler();
    root.linkWith(new TechHandler()).linkWith(new GeneralHandler());
    for (String t : new String[]{"bill:invoice", "tech:5xx", "hi"}) System.out.println(t + "->" + root.handle(t));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.chain;

public abstract class SupportHandler {
    private SupportHandler next;

    public SupportHandler linkWith(SupportHandler next) {
        this.next = next;
        return next;
    }

    public final String handle(String request) {
        var result = doHandle(request);
        if (result != null) return result;
        return next != null ? next.handle(request) : "unhandled:" + request;
    }

    protected abstract String doHandle(String request);
}

final class BillingHandler extends SupportHandler {
    protected String doHandle(String request) {
        return request.startsWith("bill:") ? "billing-ok" : null;
    }
```

Solution (2)

```
}

final class TechHandler extends SupportHandler {
    protected String doHandle(String request) {
        return request.startsWith("tech:") ? "tech-ok" : null;
    }
}

final class GeneralHandler extends SupportHandler {
    protected String doHandle(String request) { return "general-ok"; }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.chain;

import com.buildwithwaqar.patterns.gof.behavioral.chain.*;
/** Problem: route support tickets without a giant if/else in the controller. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Chain problem: support ticket routing ===");
    var root = new BillingHandler();
    root.linkWith(new TechHandler()).linkWith(new GeneralHandler());
    for (String t : new String[]{"bill:invoice", "tech:5xx", "hi"}) System.out.println(t + "->" + root.handle(t));
  }
}
```

Walkthrough Output

```
=== Chain problem: support ticket routing ===
bill:invoice->billing-ok
tech:5xx->tech-ok
hi->general-ok
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Chain.

Related Patterns

- Contrast Chain with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Chain → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.